关于在FPGA上实施项目第二阶段的框架与AI工具的讨论

项目需求

本项目是对一篇论文所提出的2个创新算法2NRM-RRNS, 3NRM-RRNS算法与传统的RRNS算法，以及RS算法进行对比，目的是证实：这两个创新性算法是否具有可实施性，从存储效率、冗余纠错能力、计算复杂度及计算资源消耗维度对四种算法进行对比，希望得出的结论是两种创新算法2NRM-RRNS,3NRM-RRNS算法是可行的，从存储效率、冗余纠错能力、算法复杂度、计算资源消耗等各个方面综合考虑，2NRM-RRNS算法是属于最佳的。 进行对比的配置为如下

- 要保存的数据为16bits（0~65535）
- 各个算法使用的模式如下表格

ECC Types	Non-Redundant, m_k	Redundant, $m_{(n-k)}$
C-RRNS	$\{2^f, 2^f - 1, 2^f + 1\}$ f=6 = {64,63,65}	{67,71,73,79,83,89}
3NRM-RRNS	$\{2^f, 2^f - 1, 2^f + 1\}$ f=6 = {64,63,65}	{31,29,23,19,17,11}
2NRM-RRNS	$\{2^f + 1, 2^f\}$ f=8 = {257,256}	{61,59,55,53}

-
- 各个算法生成的codword长度见如下表格（体现了存储效率）

ECC Type	Number of bits			Difference (%)	
	Dataword	Check word	Codeword	Compare with RS	Compare with C-RRNS
RS	4,4,4,4	4,4,4,4,4,4,4,4	48	-	
C-RRNS	6,6,7	7,7,7,7,7,7	61	+27.1	-
3NRM-RRNS	6,6,7	5,5,5,5,5,4	48	0	-21.3
2NRM-RRNS	9,8	6,6,6,6	41	-14.6	-32.8

- 各个算法在上述模式配置情况下的纠错能力对比见如下表格

Redundancy Number System	Maximum allowable fault rate for 100% data recovery
C-RRNS	7% (t = 3)
3NRM-RRNS	7% (t =3)
2NRM-RRNS	6% (t = 2)
RS-CODE	9% (t = 4)

项目第一阶段（MATLAB仿真阶段）综述

在项目的第一阶段采用了MATLAB对各个算法进行了仿真对比，主要的结论见如下：

- 不同误码率的情况下解码成功率对比见如下图

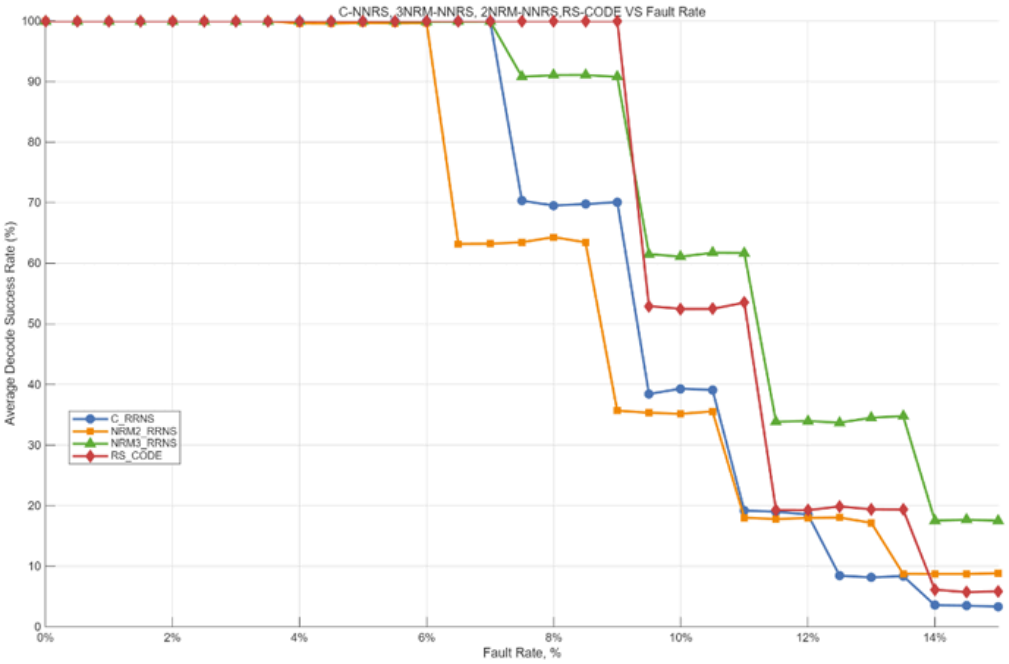

Figure 2 Average decoding success rate vs fault rate (CRRNS: MRC)

从这个图中可以看到，从有误码的情况下解码成功情况看，RS算法与3NRM-NNRS相当（3NRM-RRNS）稍微优于RS算法；C-RRNS排在中间；而2NRM-RRNS的误码情况下解码恢复能力在四种算法里面最低

- 不同解码算法的解码资源消耗（从解码时间来对比）见下图

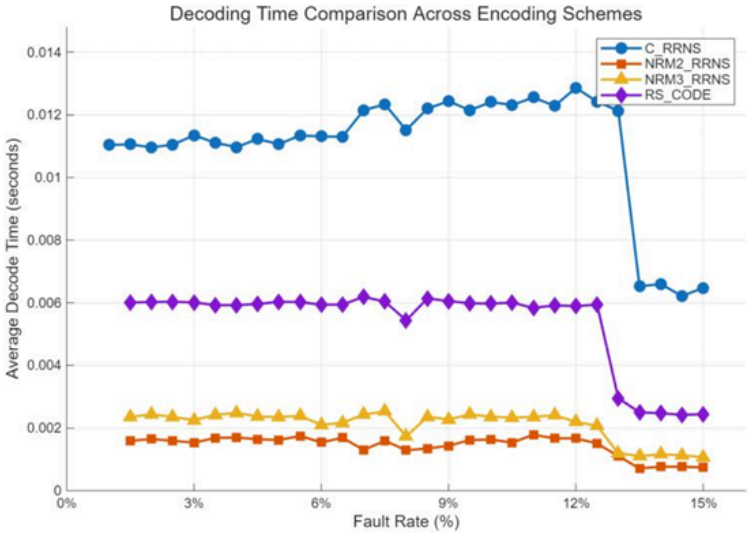

Figure 5 Average decoding latency (in MATLAB seconds) for C-RRNS using MRC decoder.

从解码资源消耗维度来看，2NRM-RRNS算法最优，其次为3NR-RRNS算法，而RS算法以及传统的RRNS算法明显比较高。

第二阶段（FPGA实现）计划要实现的目标

在第一阶段是在PC平台上使用MATLAB进行仿真对比，在第二阶段打算在FPGA平台上（目前选择的是ARTY A7-100T)对四种算法进行算法实现，然后对一定样本数据，利用四种编码算法进行编码，然后在不同误码率（从1%误码到10%误码），以及不同误码模式（均匀的

random bit反转误码，以及连续的cluster误码）进行解码，对照四种算法的解码成功率，解码速率，并对比四种编解码的资源效率（对比资源消耗，可以支持的最高时钟频率）

第二阶段的顶层实现设想

对于第二阶段的顶层设想，我初步考虑是

- step1：实现UART接口，可以通过PC控制FPGA开发板，进行指令下发，以及搜集FPGA运行结果数据搜集。目前这步基本已经实现
- step2：先实现某个算法（优先为2NRM-RRNS）的编码和解码。目前意见初步实现，包括进行了初步simulation。
- step3：集成UART，以及2NRM-RRNS算法的编码和解码，实现loopback测试，对该算法的编码和解码进行测试，验证无误码、一个码字误码、2个码字误码、3个码字误码情况下的测试。目前该步骤刚刚启动，前期是使用3NRM-RRNS来做这一步的，但3NRM-RRNS算法太复杂，导致时钟受限无法支持100MHz，需要进一步优化芯片实现才能支持100MHz，因此需要调整使用2NRM-RRNS算法实现这个。
- step4：实现如下几个模块，完成一个算法的全流程验证：
  - 在PC上利用MATLAB或python脚本产生测试数据（包括10000组在0~65535范围内的随机测试数据）作为要进行对比测试的原始样本
  - 把测试数据存储到FPGA开发板的特定存储区域，作为后续测试基础
  - 实现利用2NRM-RRNS算法对测试数据做编码，存储在FPGA开发板的特定存储区域
  - 实现加误码算法，按照拨码开关设定的误码率以及误码模式（random bit错误或者cluster错误）对上述输出的编码区域进行破坏
  - 利用2NRM-RRNS解码对上述加入了误码的数据进行解码，在UART串口输出解码成功率，解码花费的时钟周期（作为解码效率对比依据）
- step5：当上述步骤完成以后，就继续实现3NRM-RRNS,传统的RRNS,RS几种算法的编码和解码，分别输出数据
- step6：当上述步骤都可以完成，做进一步的优化，包括两个优化
  - 优化点1：在FPGAG开发板的硬件资源可以支撑的前提下把步骤4通过拨码开关设置误码率以及误码模式优化掉，直接自动完成bit random以及cluster（设置3个连续bits）不同误码率的加误码和解码并输出结果
  - 优化点2：在FPGA开发板的硬件资源可以支撑的前提下，把上述步骤2-5都集成到单板上，启动以后可以自动完成全部对比测试，在串口把各种模式都完成测试并输出解码信息